

IRONUM · PRACTICAL GUIDE

AI for product owners

Where it fits your sprint, and where it stops.

Of German workplaces already using generative AI, 27% give their people any training in it. Nine in ten use freely accessible tools.

Alesia Kunz, Product Manager at Ironum. 17+ years in software engineering as a product manager and product owner. Built on experience from real teams and projects, and on published research.

Inside: the connector that works and still returns nothing, the one part of the job no tool can take, and the prompts worth keeping.



Contents

The training barely exists

Part 0: six things nobody told you

1. Your best prompt might have been luck
 2. Your Copilot probably cannot see your work
 3. Microsoft's four-part prompt
 4. The slash key
 5. Where your prompts actually go
 6. It will be wrong in your own house style
-

Part 1: four habits that separate method from luck

- Rule 1. Give it your own material
 - Rule 2. Know what must never go in
 - Rule 3. Check it before it leaves you
 - Rule 4. Keep what works
-

Part 2: the five things that are actually yours

- Chapter 1. Why your items come back from refinement
 - Chapter 2. Refinement is not a group reading session
 - Chapter 3. Ordering is the part nobody can do for you
 - Chapter 4. Planning starts before the meeting does
 - Chapter 5. What got done, and what actually changed
 - Back to the obstacle
-

Part 3: the prompt pack

Part 4: staying current on thirty minutes a week

- Where to look, and how often
 - A weekly on AI news
-

When the limit is the tool, not you

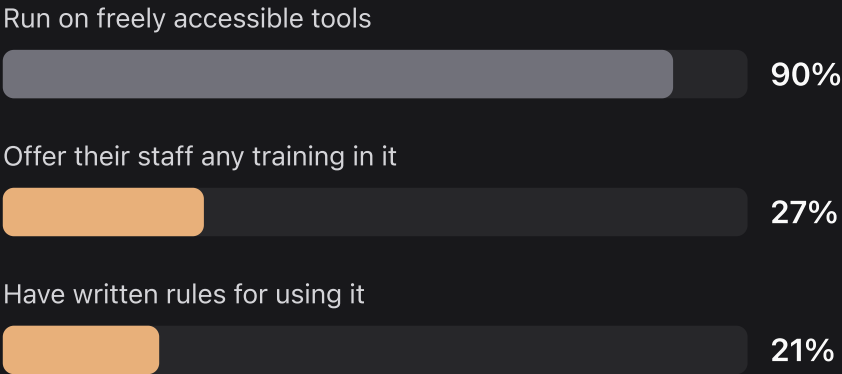
Sources

- Notices

Only 21% of those workplaces have written down any rules for using it. So in most of them the tool is in the building, nobody was taught it, and nobody agreed what may be typed into it.

Everyone is using it. Almost nobody set it up

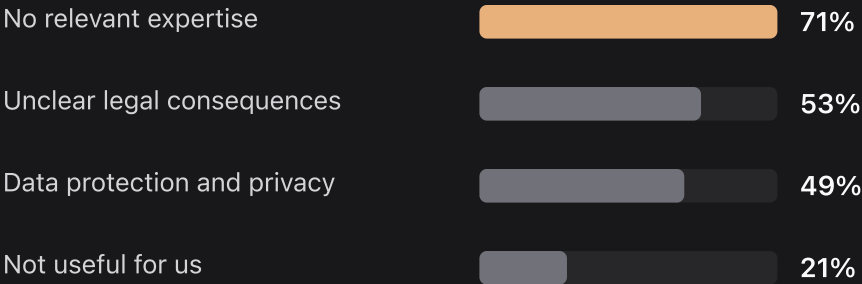
Of establishments that already use generative AI



Those figures come from the IAB-Betriebspanel, the establishment panel run by the research institute of the German Federal Employment Agency. Everything about the role itself comes from the Scrum Guide, everything about Copilot from Microsoft's own documentation, and the European and training figures from Eurostat and the OECD.

What actually stops people

EU enterprises that considered AI, 2025



Missing expertise is the reason named most.

The training barely exists

If nobody has trained you, that is normal. The OECD looked at the training actually on offer across Australia, Germany, Singapore and the United States and found that between 0.3% and 5.5% of courses deliver any AI content at all. Most of the public programmes the OECD reviewed

focus on advanced AI skills rather than everyday use. The OECD notes the share may be underestimated, since some learning happens on the job.

What the tool is, in two sentences: a system that predicts likely text from the text you give it, trained on an enormous amount of writing. That is why it writes so smoothly, why it helps with anything made of words, and why it will state something false with complete confidence.

Start here, once, this week. Take a backlog item you wrote yourself and know well. Ask it to list the questions a developer would ask before picking the item up. Because you wrote the item, you can judge the answer straight away.

Part 0: six things nobody told you

Six short sections before the main guide, a minute each. Most of them explain something that goes wrong and is not your fault.

1. Your best prompt might have been luck

Microsoft says it plainly in its own guidance: using the same prompt multiple times can result in different responses. That is how the technology works.

One good answer does not prove the prompt was good. It might have been luck. A prompt is only worth keeping once it has produced a usable answer more than once, on different inputs. That is why Rule 4 exists.

2. Your Copilot probably cannot see your work

There are three tiers, and they reach very different things.

What each Copilot can reach

Same name, three different products

	WEB	IN APPS	YOUR FILES
Copilot Chat (Basic)			
Microsoft 365 Copilot (Basic)			
Microsoft 365 Copilot (Premium), paid add-on			

An empty cell means it sees your work only if you paste or upload it.

The same thing in detail, including what each tier can reach and how:

TIER	CAN IT SEE YOUR FILES AND MAIL
Copilot Chat (Basic)	No. Web data only. It sees your work only if you paste it, upload it, or have it open in Teams or Outlook
Microsoft 365 Copilot (Basic)	Not in chat. But Copilot works inside Word, Excel, PowerPoint and OneNote
Microsoft 365 Copilot (Premium), the paid add-on	Yes, automatically, through Microsoft Graph, and only for files you already have permission to open

Microsoft is renaming Microsoft 365 Copilot to Microsoft Copilot, and Copilot Chat to Microsoft Copilot Chat, so your screen may show either name.

If you have asked Copilot "what did we agree about this epic" and got nothing useful, check your tier before you blame your prompt. Only the premium tier reaches your own content by itself.

The model picker. Copilot offers Auto, Quick response and Think deeper. Auto chooses for you. Switch to Think deeper when the task is a judgement rather than a lookup, and expect it to take longer on purpose.

Grounding, which is the word for all of this. Two sources feed any answer: the public web, and your organisation's own content. Only the premium tier reaches the second one by itself. Every other tier sees only what you hand it.

Your backlog probably is not in Microsoft 365. If it lives in Jira or Azure DevOps, no tier of Copilot reads it automatically. Microsoft does publish connectors for Jira and Confluence, but an administrator has to deploy them and choose whether they respect Jira's own permissions. There are separate connectors for Jira Cloud and Jira Data Center, and none for Jira Server. A connected source is not the same as a readable project.

Field note. Some Copilot licences offer the Jira integration, and you can check yours under Copilot Chat, then Settings, then Sources. The catch is that even where Jira or Confluence shows up there as an available source, Copilot often still cannot pull data from your particular project. In practice that means product owners and business analysts copy the ticket in by hand to refine a story or its acceptance criteria. Before you paste anything into Copilot or any other AI tool, check that your organisation allows it from a data protection point of view. That check comes before the convenience.

3. Microsoft's four-part prompt

This framework comes from Microsoft's own guidance, and it is a good one.

Goal. What you want. **Context.** The situation, who it is for, what has already been decided. **Expectations.** Format, length, tone, what not to do. **Source.** Where it should get the material.

A thin prompt: *Split this story.*

The same prompt with four parts: *Split the story below into smaller items. Context: a B2B web product, the team delivers in two-week sprints, and this story is too large to finish in one. Expectations: each item independently valuable and testable on its own, with the acceptance criteria that would prove it. Say which split you would not recommend and why. Source: only the story below.*

The second one is just more specific about what you already know.

4. The slash key

In Copilot, type  and start typing the name of a file. With the paid Microsoft 365 Copilot licence you can also reference people, meetings

and emails. You can attach a single file or a whole folder. Inside a SharePoint site you can reference up to ten files or pages. Checked September 2026.

5. Where your prompts actually go

When you are signed in with your work account, prompts and responses are covered by enterprise data protection, and Microsoft states they are not used to train the foundation models. Copilot cannot show you a document you could not already open yourself. For users in the European Union there are additional European Union Data Boundary safeguards, with two exceptions: web search queries, and Anthropic models where your administrator has enabled them, are not covered by the EU Data Boundary.

The caveat that matters. All of that describes a work account signed in with your company identity. A personal account is a different product with different terms.

In Eurostat's survey of EU enterprises, 49% of enterprises that considered AI and did not adopt it named data protection and privacy, and 53% named unclear legal consequences. Respondents could name more than one. This section answers part of both. This is practical guidance, not legal advice, and your company's own rules sit on top of it.

6. It will be wrong in your own house style

It will state things that are not true, and it will state them confidently. Three things need checking every time:

- **Numbers.** Check every figure you did not give it yourself.
- **Names.** People, products, teams, systems.
- **Anything stated as a fact about a user or a customer.** This is the dangerous one, because it is the hardest to spot and the most expensive to put in a backlog.

What this looks like in practice. You paste thirty support tickets and ask for the themes. Back comes: "Customers consistently request single sign-on." You go looking, and one ticket mentioned it once, as an aside. Nothing was invented exactly. A single signal was promoted to a pattern, in confident language, in a sentence that could become an epic. That is the failure to expect.

One mention is not a pattern

The failure mode to expect when you ask for themes

30 SUPPORT TICKETS PASTED IN



1 ticket

mentioned single sign-on,
once, as an aside

WHAT CAME BACK

"Customers consistently request single sign-on."

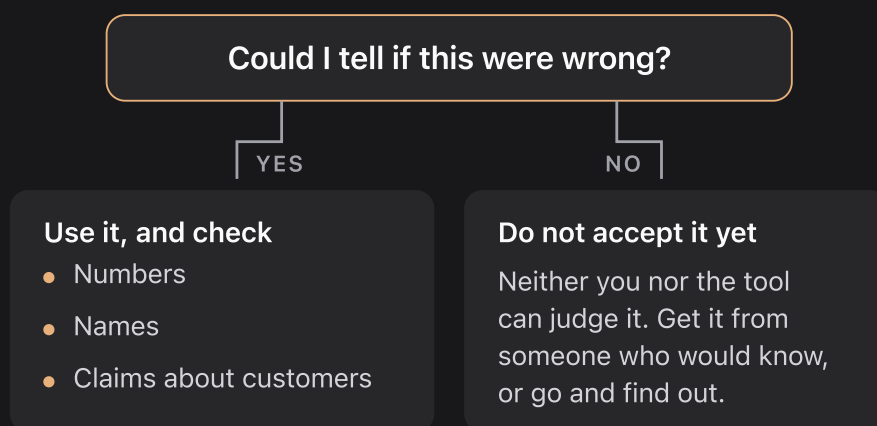
One ticket in thirty, reported as consistent.

Field note. Our rule for any number an AI tool hands us: if we cannot trace it to its source, we remove it. It does not matter how plausible it looks. It always looks plausible.

The decision aid. Before you use an output, ask one question: *could I tell if this were wrong?* If yes, use it and check it. If no, do not use it yet. Get the answer from someone who would know, or go and find out.

Before you use the output

One question, asked every time



Part 1: four habits that separate method from luck

Part 0 was about how the tool behaves. These four habits are about how you work with it, and they are what makes a good result repeatable.

Rule 1. Give it your own material

The difference between a generic answer and a useful one is almost always what you put in, not how you phrase the question.

Paste the actual ticket, not your summary of it. The actual transcript, not your memory of the refinement session. The actual acceptance criteria you rejected, not a description of them.

Rule 2. Know what must never go in

Three questions you can answer in a second:

1. **Which account am I in?** Work identity, or personal. See Part 0, item 5.
2. **Would I put this in an email to a supplier?** If not, it does not go in a chat window either.
3. **Is it someone else's personal data?** User research recordings, support tickets with names in them, anything from a customer system. Anonymise or do not paste.

Rule 3. Check it before it leaves you

Check the same three things every time: numbers, names, and anything stated as a fact about a user. The list is in Part 0, item 6.

The moment that matters is **before it leaves you**, not before it ships. Once a plausible wrong sentence is inside a backlog item, it stops being an AI problem and becomes something a team builds.

Rule 4. Keep what works

When an answer comes out good, save the prompt that produced it, with the parts that change marked in square brackets, wherever your team already looks. Next time the same job starts from something that has already worked, instead of from nothing.

If your team has no shared place for prompts yet, make one. Saved prompts are how this becomes a habit rather than a one-off.

Part 2: the five things that are actually yours

Rules are easy to agree with and easy to forget. Here they are against the product owner's work as the Scrum Guide describes it, in the order the work happens: from writing an item to showing what it changed.

Chapter 1. Why your items come back from refinement

The need. An item goes into refinement and the session is spent filling in things you could have written down first.

Where it helps.

- Turning a rough note into a first item you then rewrite, which is faster than facing an empty field.
- Listing the questions a developer would ask before picking the item up, which is the fastest way to find what you left out.
- Rewriting an item that got bounced in refinement, using the objections as input.

What it cannot do. It cannot know why the item matters. That is the part you supply, and it is the part that makes the item worth building.

One prompt. Works on any tier, because you paste the content in.

```
Goal: list the questions a developer would ask before starting the
backlog item below.
Context: [product], [team], the item is meant to fit inside one
sprint.
Expectations: the questions only, at most ten, ordered by how much
the answer could change the size of the item, each tagged big, small
or unknown, with unknown ones placed before small ones, so I can
check the order. Do not answer them and do not rewrite the item.
Source: only the item below.
```

```
[paste the item]
```

Field note. These tools save a serious amount of time on the parts of an item you should have settled before the session. They do not remove the questions, and they should not. Where the team works well and the goal is clear, good questions still come up about the aspect nobody considered, and that is exactly right. Writing a strong epic, feature or story is teamwork. The tool clears the avoidable questions so the session can reach the ones worth having.

Your first step. Run it on the item currently at the top of your backlog. Take anything on that list you cannot answer into refinement as a question for the team, rather than discovering it there.

Chapter 2. Refinement is not a group reading session

The need. Refinement becomes forty minutes of the team reading a story for the first time.

Where it helps.

- Proposing splits for an item that is too large, which you then accept or reject.
- Drafting acceptance criteria that are actually testable rather than restatements of the title.
- Finding the assumption buried in a story, the thing everyone reads past.

What it cannot do. It cannot tell you which split delivers value on its own. It will happily split by technical layer, which produces items nobody can ship.

One prompt.

```
Goal: propose three different ways to split the story below into
smaller items, each cutting along a different line.
Context: [product]. Each resulting item must be independently
valuable to a user and finishable inside one sprint, so do not split
by technical layer.
Expectations: for each split, the resulting items in one line each,
and what a user could do after the first one alone. Flag any item
that breaks a rule in Context. Then name the split you would not
recommend and say why.
Source: only the story below.

[paste the story]
```

Your first step. Take the largest item in your backlog and run it. The split you would not recommend is often the one a team reaches for first.

Chapter 3. Ordering is the part nobody can do for you

The need. What sits at the top is there because someone pushed hardest, and you are the one who has to defend it anyway.

The Scrum Guide is explicit here: you may delegate the work but remain accountable, and the product owner is one person, not a committee.

Where it helps.

- Arguing the other side of an ordering decision before a stakeholder does it for you.
- Making the trade-off explicit, so what you are giving up is written down rather than discovered later.
- Checking an order against the Product Goal, which the Scrum Guide defines as a future state of the product that the team can plan against, and naming the items that no longer serve it.

What it cannot do, and this is the important one. It cannot order your backlog. A model that has never met your users, does not know how much change your organisation can take and has not sat in the conversation is guessing, fluently. A draft from a model does not take that accountability off you. Use it to test an order you already hold, never to produce one.

Field note. What actually wins an argument about order is analytical data behind the position, and this is where AI is genuinely useful: finding validated sources that support building a feature, or that show the assumption behind it was wrong and it should be dropped. Both directions count. On one project the assumption was that all customers were asking for AI avatars in the platform. When the data was analysed in detail, only 10% of users had asked for it, and 90% said it was nice to have. The feature was deprioritised. The value is in being willing to lose the argument to the data.

The specific danger. Generating backlog items in bulk. It is the easiest thing to do with these tools and the least useful. Every item you add is something the team must read, estimate and eventually delete.

One prompt.

Goal: make the strongest case against the order below.
Context: the Product Goal is [goal]. These are the next [N] items in the order I have put them in: [items].
Expectations: up to three of the strongest objections to the order of the items that serve the goal, each naming what would have to be true for the objection to win. Then list any item that does not serve the Product Goal at all, or say none if they all do.
Source: only what I have written.

Your first step. Run it before your next stakeholder conversation about priorities, not after.

Chapter 4. Planning starts before the meeting does

The need. Planning opens with people picking tickets and nobody has said what the sprint is actually for.

The first topic of sprint planning is why this sprint is valuable, and the Scrum Guide makes proposing that value your job.

Where it helps.

- Turning a set of selected items into a candidate sprint goal in one sentence, which you then correct.
- Asking whether the selected items actually hold together, or whether they are just the next things in the list.

What it cannot do. It cannot decide what is valuable. A model that has never met your stakeholders is guessing. What it can do is make you articulate the goal before the meeting, so you arrive with a sentence instead of a list.

One prompt.

```
Goal: draft a candidate sprint goal from the items below.  
Context: [product], a [length] sprint. The Product Goal is [goal].  
Expectations: one sentence naming the outcome a user or the business gets, in terms of the Product Goal. It must not name or reword any item, feature or component from the list: say what is different for the user afterwards, not what the feature lets them do. If most items belong to one feature and the outcome cannot be stated without it, say so and name it rather than writing something vague. Then say which items do not contribute to that goal, and whether the items that do contribute hold together as one objective or are really two.  
Source: only the items below.
```

```
[paste the selected items]
```

Your first step. Draft it before planning and bring it as a proposal. The Scrum Team defines the sprint goal together, and a draft gives that conversation somewhere to start.

Chapter 5. What got done, and what actually changed

The need. The room stays quiet, because nobody can say what is actually different for a user.

Where it helps.

- Turning what was built into what changed for a user, which is a different sentence and the one stakeholders need.
- Preparing for the questions you will be asked, including the uncomfortable one about what slipped.
- Listing what the sprint did not settle, which the standard recap tends to leave out.

What it cannot do. It cannot demonstrate the increment. The Scrum Guide is blunt about this: the sprint review "is a working session and the Scrum Team should avoid limiting it to a presentation." A polished narrative that hides a thin increment costs you more than it buys.

One prompt.

```
Goal: turn the completed items below into what changed for a user.
Context: [product], [stakeholders and what they care about]. The
Product Goal is [goal].
Expectations: one line per item saying what someone can now do that
they could not before. Mark any item where you cannot tell, rather
than guessing. Then list the three questions a sceptical stakeholder
would ask.
Source: only the items below, plus the stakeholders I named in
Context. This applies to the questions as well: do not reference any
story, ticket, discussion or prior release that is not listed here.

[paste the completed items]
```

Your first step. Run it on your last completed sprint. Any item where it cannot say what changed for a user is worth a conversation with the team.

Back to the obstacle

The largest thing standing between organisations and AI is a lack of relevant expertise, named by 71% of the EU enterprises Eurostat surveyed that considered AI and did not adopt it, well ahead of anything else.

The five chapters answer that, and the answer is deliberately plain. Nothing above asks you to adopt a new practice, add a ceremony or change how your team works. Each one sits inside work the Scrum Guide already gives you or your Scrum Team: writing items, refining them, ordering them, proposing the value of a sprint, showing what changed.

That is also why none of it works without you. The tool drafts and argues. You decide which split delivers value, whether an objection to your order wins, and whether the sprint goal holds together.

Part 3: the prompt pack

Start a new chat for each prompt, so nothing from an earlier task leaks into the answer, and on a fast or free model check the output against the prompt's own rules before you use it.

Every prompt below uses the four parts from Part 0, item 3, though the in-app prompts skip Context, because the open thread or meeting supplies it. Square brackets mark the parts you change. Each says what it needs, and "any tier" means it works even on Copilot Chat (Basic), because you supply the content yourself.

BACKLOG HYGIENE, ANY TIER

Find what should not be there.

Goal: review the backlog items below against the Product Goal.
Context: the Product Goal is [goal].
Expectations: four lists. Items that clearly serve the goal, items that do not, items whose fit is arguable with three words on why, and items that are duplicates or near-duplicates of each other. An item may appear in more than one list. Quote the item titles, do not paraphrase them.
Source: only the items below.

Find the items nobody can start.

Goal: list the backlog items below that have no testable acceptance criteria.
Expectations: the item titles only, and for each every category that is missing: scope, measurable outcome, or pass/fail condition. Name the categories only. Do not give an example of the criterion.
Source: only the items below.

Find the buried assumption.

Goal: name the assumptions the story below depends on.
Context: [product], [who the user is].
Expectations: assumptions only, each with how we could check it cheaply. Do not propose solutions.
Source: only the story below.

TEAMS, NEEDS MICROSOFT 365 COPILOT

Copilot in Teams reaches back over the last 30 days of meetings, which is easy to miss.

Turn a refinement session into decisions.

Goal: extract what this session decided about each item.
Expectations: a table of item, decision, open question. Every item gets a row; leave either column blank where there is nothing to put in it. Then list the items with a blank decision again under a heading titled undecided. Do not invent decisions.
Source: this meeting only.

OUTLOOK, NEEDS MICROSOFT 365 COPILOT

Find the request buried in a long thread.

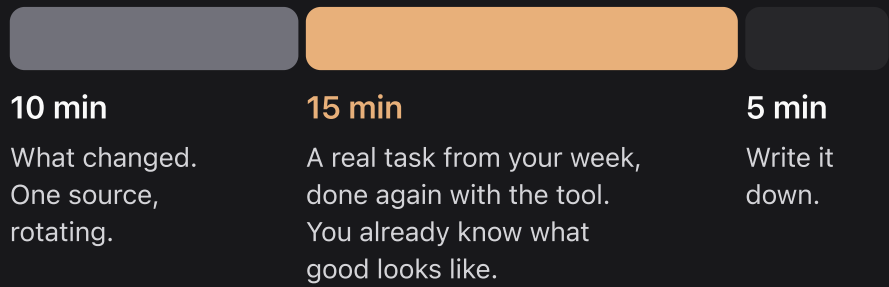
Goal: tell me what, if anything, this thread asks me to change in the backlog. If it asks for a commitment, a date or an estimate instead, say that, and do not invent a backlog item or say what is already in it.
Expectations: the request in one line, everyone asking for it, what they say the value is, and whether they have said anything about urgency.
Source: this thread only.

Part 4: staying current on thirty minutes a week

Waiting for a course will not close the training gap at the top of this guide. On the OECD's count, between 0.3% and 5.5% of courses carry any AI content. Here is a routine that costs nothing and needs nobody's approval.

Thirty minutes a week

No budget, no approval, no course



The middle block is the one that teaches. The other two make it stick.

Ten minutes, what changed. One source from the list below, rotating through it.

Fifteen minutes, on your own real work. Take something you did this week, a split, an item, a sprint goal, and do it again with the tool. Not an exercise, the actual work, where you already know what good looks like.

Five minutes, write it down. One line in your saved prompts: what worked, what did not. This is Rule 4.

Where to look, and how often

QUESTION	SOURCE	HOW OFTEN
What changed in Copilot	Microsoft 365 Copilot release notes on Microsoft Learn	Monthly
What changed in the models	The providers' own release notes	When something ships
What changed in the framework	The Scrum Guide at scrumguides.org, which is versioned and short	When a version lands
What changed in the rules, in the EU	The European Commission's AI Act pages. The AI literacy duty in Article 4 has applied since 2 February 2025	Quarterly

Do not treat vendor blogs, news aggregators or social threads as primary sources. Go back to whoever published the number.

A weekly on AI news

If you would be interested in a weekly newsletter on AI news, write to us at info@ironum.com.

If this guide was forwarded to you, the other one is at ironum.com/resources/guides.

When the limit is the tool, not you

Most of this guide is method, and method is yours to fix. Some limits are not. You have met four of them already:

- If your backlog lives in Jira or Azure DevOps, no tier of Copilot reads it automatically (Part 0, item 2).
- Microsoft's Jira connectors cover Jira Cloud and, separately, Jira Data Center, not Jira Server, and even a connected source is often not a readable project.
- So in practice, product owners copy tickets in by hand. Every prompt in this guide starts from what you paste.
- Web search queries and Anthropic models sit outside the EU Data Boundary (Part 0, item 5).

When a model can read your backlog directly, those limits move. It can check every item against the Product Goal, find duplicates across the whole backlog rather than the twenty items you pasted, and refine a story against its real history, while respecting who may see which project. That is called grounding, and it does not have to mean sending your data abroad: the model can run on servers in Germany, and it can be a model you own.

This is the software engineering work Ironum does: connecting AI to the systems a product team already uses, Jira and Confluence included, and hosting it in the EU or on your own infrastructure.

The technique is called enterprise RAG: the model answers from an organisation's own material and cites each answer back to its source (ironum.com/services/enterprise-rag).

If you have hit one of these limits, write to info@ironum.com with the limit and the system your backlog lives in. Alesia replies herself, within

two working days, with whether it is solvable and roughly what it would take. No sales sequence follows.

Sources

Every source below was checked on 17 September 2026.

- **The Scrum Guide**, by Ken Schwaber and Jeff Sutherland, for every statement about what a product owner is accountable for and what happens in each Scrum event. Licensed under Creative Commons Attribution ShareAlike 4.0. scrumguides.org/scrum-guide.html
- **IAB-Betriebspanel 2025**, the establishment panel run by the research institute of the German Federal Employment Agency, published as IAB-Kurzbericht 8 of 2026, for the figures on who uses generative AI, who trains people in it, who has written rules for it, and what share run on freely accessible tools. doku.iab.de/kurzber/2026/kb2026-08.pdf
- **Eurostat**, use of artificial intelligence in enterprises, 2025 data, for why enterprises that considered AI did not adopt it. 157,000 of the EU's 1.53 million enterprises surveyed. ec.europa.eu/eurostat/statistics-explained, dataset `isoc_eb_ai`
- **OECD**, Bridging the AI skills gap: is training keeping up, April 2025, for how little of the available training carries any AI content. oecd.org/en/publications/bridging-the-ai-skills-gap_66d0702e-en.html
- **Microsoft**, product documentation, for Copilot licence tiers, grounding, data protection, file referencing and prompt structure, and for the Jira and Confluence connectors: an administrator must deploy them and set their permissions, and Jira Cloud and Jira Data Center have separate connectors (checked 8 October 2026). learn.microsoft.com/microsoft-365/copilot/microsoft-365-copilot-overview, [/copilot/connectors/jira-cloud-overview](https://learn.microsoft.com/microsoft-365/copilot/connectors/jira-cloud-overview) and [/copilot/connectors/jira-data-center-overview](https://learn.microsoft.com/microsoft-365/copilot/connectors/jira-data-center-overview)
- **European Commission**, for the AI Act and the Article 4 AI literacy duty applicable since 2 February 2025. digital-strategy.ec.europa.eu/en/faqs/ai-literacy-questions-answers

No figure here is estimated, modelled or taken from a vendor's marketing material.

Notices

The Scrum Guide is by Ken Schwaber and Jeff Sutherland, published at scrumguides.org under the Creative Commons Attribution ShareAlike 4.0 licence, creativecommons.org/licenses/by-sa/4.0. Quoted here with attribution; this guide is not endorsed by or affiliated with its authors.

Microsoft, Microsoft 365 and Copilot are trademarks of Microsoft. Jira and Confluence are trademarks of Atlassian. Other product names are the marks of their owners and are used here only to refer to the products themselves. No affiliation or endorsement is implied by any of them.

Figures from third-party research are quoted with attribution to make a point, and no chart from any source is reproduced. Every graphic here was drawn from the cited numbers.

Nothing in this guide is legal advice. Your organisation's own policies govern what you may put into any tool.



Alesia Kunz, Product Manager at Ironum

Also CEO of LearnSlice, the company behind Ironum. 17+ years in software engineering as a product manager and product owner. This guide is built on two things: experience from real teams and projects, and published research. The field notes are the experience. Every figure is traced to the organisation that published it, and every source is listed above. The two are styled differently on purpose, so you can always tell which you are reading.

If you have a question. Whether it is about applying AI to your own working week, or about support with software engineering, write to me at info@ironum.com or find me on LinkedIn at linkedin.com/in/alesiakunz.